

Model 2 → Cesium View

What the user sees →

- **Default view:** opens directly on the **survey location**; only the reconstructed survey model is shown.
- The whole globe is **not visible by default**.
- When **Whole Globe / Global Context** is enabled, the 3D globe opens and the survey model remains positioned at its **actual geographic location**.
- Satellite imagery / world terrain provides geographic context.
- **Only the surveyed area is a 3D reconstruction**; the rest of the globe remains a **2D/context view**, not a reconstructed 3D model.
- High-resolution reconstructed terrain, buildings, roads, vegetation and other surveyed structures are visible.
- Flight trajectory can be displayed.
- Survey boundaries can be displayed.

What the user can do →

- Rotate, pan, zoom and navigate around the survey model.
- Enable **Whole Globe / Global Context** and navigate around Earth.
- Move between global context and the survey location.
- Toggle:
 - Globe/world terrain
 - Satellite imagery
 - Survey reconstruction
 - Flight trajectory
 - Survey boundary
- Select buildings, roads, terrain or other reconstructed geometry.
- **Area measurement from above:** select multiple points to create a polygon.
- Choose the **polygon/sheet color**.
- Adjust polygon opacity so the underlying terrain remains visible.
- Clear, add or reposition measurement points.

What happens when they click / select something →

- **Terrain:** show latitude, longitude and elevation.
- **Building:** show building information, dimensions, height, footprint and elevations.
- **Area selection:** each clicked point becomes a georeferenced measurement point; points are connected into a polygon.
- The selected polygon appears as a **thin, semi-transparent colored sheet** over the survey area.

- The system calculates the area and perimeter from the georeferenced coordinates.
- The selected geographic points are linked to their corresponding **Point IDs / Object IDs** in the other views.

Measurement & Inspection workflow →

Area measurement is the primary measurement tool in Cesium View.

1. The user switches to **Area Measure**.
2. The user clicks points from the **top/above view** around the required location.
3. The system creates the polygon boundary.
4. A semi-transparent colored sheet covers the selected area.
5. Users can change sheet color and adjust opacity.
6. System calculates:
 - **Area** — m² / hectares
 - **Perimeter** — m
 - **Number of selected points**
 - **Minimum elevation**
 - **Maximum elevation**
 - **Average elevation**
 - **Elevation difference**
7. Measurement uses the **georeferenced metric geometry**, not screen pixels.

For object inspection:

- **Terrain** → Latitude, Longitude, Elevation
- **Building** → Height, Footprint Area, Length, Width, Base Elevation, Roof Elevation
- **Two selected locations** → Distance and elevation difference

Layer / Filter controls →

- **Globe / World Context**
- **World Terrain**
- **Satellite Imagery**
- **Survey Reconstruction**
- **Flight Trajectory**
- **Survey Boundary**
- **Area Measurement**
- Optional visibility/opacity controls for applicable layers.

How switching between modes works →

- Switching between **Cesium, Point Cloud and Semantic View** does **NOT** start **reconstruction again**.
- All required assets and mappings are generated during the original reconstruction job.

- Switching is therefore a **visualization/data-loading operation only**.
- Cesium loads the precomputed georeferenced survey model and associated spatial data.
- Point Cloud loads the precomputed point cloud.
- Semantic View loads the precomputed classified 3D scene.

How the three modes stay synchronized →

Use a common **spatial identity system**:

Point ID → Object ID → Geographic XYZ → Semantic Object → Cesium Geometry

When the user selects an **area in Cesium**:

- The selected polygon points retain their geographic coordinates.
- Corresponding points/regions are found in the Point Cloud using the spatial index.
- The corresponding **points/region are highlighted in Point Cloud View**.
- In Semantic View, the corresponding **terrain and structures intersecting the selected area are highlighted**.
- Buildings/objects inside the selected area can be highlighted using their Object IDs.
- The synchronization is based on **coordinates + Point IDs + Object IDs**, not visual approximation.

This should work in the reverse direction as well:

- Point Cloud selection → corresponding objects/terrain highlighted in Semantic View.
- Semantic selection → corresponding geometry/location shown in Cesium.

Is this possible? Best approach →

Yes, this is achievable. For your priorities, use **accuracy first, then processing speed**.

≤1 m accuracy: achievable as a **validated project target**, but it cannot be guaranteed simply from ordinary drone GPS + RGB video. To realistically approach ≤1 m, the pipeline should use:

- Metric-scale 3D reconstruction.
- Accurate camera calibration/intrinsics.
- Accurate video/GPS timestamp synchronization.
- High-quality GPS/telemetry.
- GPS-camera trajectory alignment.
- **Umeyama similarity alignment** for scale, rotation and translation.
- Independent checkpoints/GCPs or RTK/PPK where available.
- Reconstruction confidence filtering.
- Final RMSE / median / 95th-percentile accuracy validation.

The system should **report the measured accuracy**, rather than claiming every reconstruction is ≤ 1 m.

<15 min for a 10-minute video: achievable as an **engineering target**, but it must be benchmarked on representative hardware/data. The main strategy is:

- Adaptive keyframes instead of processing every frame.
- AMB3R-SfM for reconstruction.
- Parallel preprocessing and asset generation.
- Efficient point-cloud fusion/indexing.
- Selective depth refinement only in weak regions.
- GPU acceleration.
- Precompute all three visualization assets during the reconstruction job.

Requirement in model →

Add/ensure the following:

Phase 2 → Data Preparation

- GPS-aware candidate frame selection.
- Frame quality scoring.
- Shot/camera-motion analysis.
- Visual overlap and spatial-baseline checks.
- Video–GPS synchronization.
- Dynamic object masking.

Phase 3 → 3D Reconstruction

- AMB3R-SfM metric reconstruction.
- Camera poses/trajectory.
- Metric depth.
- Dense pointmap generation and fusion.
- Reconstruction confidence.
- Spatial consistency/quality checks.

Phase 4 → Scene Understanding

- Semantic segmentation.
- 2D → 3D semantic projection.
- Persistent **Object IDs**.
- Dynamic-object removal.

Phase 5 → Object & Terrain

- DTM/DEM generation.

- Terrain mesh.
- Building footprints/heights.
- Road/vegetation geometry.
- Measurement-ready geometry.
- Point → Object association.

Phase 6 → Refinement & Georeferencing

- WGS84 → ECEF → local ENU.
- GPS-camera alignment.
- Umeyama transformation.
- Georeference all geometry.
- Accuracy/QA metrics.
- Selective depth refinement.
- Mesh generation.

Phase 7 → Output

- Georeferenced GLB/3D Tiles for Cesium.
- LAS/LAZ/PLY for Point Cloud.
- Classified 3D geometry for Semantic View.
- DEM/DTM.
- `metadata.json`.
- Persistent Point IDs/Object IDs.
- Spatial index.
- Cross-view mapping.
- CRS, units, transformation and accuracy metadata.

Backend & model flow during loading →

Since **Point Cloud is the first major reconstruction asset**, the backend should build the shared spatial foundation immediately:

Input → Preprocessing → Adaptive Keyframes → AMB3R-SfM → Metric Depth + Camera Poses + Pointmaps → Pointmap Fusion → Confidence/Outlier Filtering → GPS/Telemetry Alignment → WGS84/ECEF/ENU + Umeyama → Georeferenced Point Cloud

Then immediately prepare:

- Point ID
- XYZ
- Latitude/Longitude
- Point elevation
- Terrain elevation
- RGB

- Semantic class
- Semantic confidence
- Reconstruction confidence
- Object ID
- Spatial index
- Point ↔ Object mapping
- DEM/DTM
- Terrain mesh
- Survey boundary

In parallel, backend prepares:

Point Cloud → LAS/LAZ + spatial index

Semantic data → classified 3D geometry + Object IDs + semantic attributes

Cesium data → georeferenced GLB / 3D Tiles + survey boundary + geographic positioning

Common metadata → CRS + transformations + camera trajectory + QA + cross-view mappings

Therefore, when the job reaches **100%**, all three views are already prepared. The frontend only loads/streams the required assets, so **switching views does not trigger ML reconstruction again**.